

Delta Live table (DLT)

Databricks

what is DLT?

A framework to Build ETL pipelines with a Declarative approach

what is Declarative approach?

- we tell what to do (not how we do it)

SCDtype 2

Simplify traditional ETL

traditional ETL and its challenges

=====

- how to extract the data

- how to transform the data

- how to load it

Orchestration framework to stitch things together.

Infra management, scaling up and scaling down the infra

manage dependencies

track lineage

handle batch and streaming in the pipeline

ensure data quality

handle failure and retries

monitor and optimize pipeline

deploy in multiple env

time spent on tooling kind of dominate your efforts.

DLT solves the above issues, and make it easy to build ETL pipelines.

why declarative programming?

understanding your objective allows DLT to take care of all the boring stuff.

1. we write the declarative code (SQL or Python)

2. we create a DLT pipeline to run the DLT code

Demo 1

=====

Orders- Transactional Data

Customers data- Master Data

Medallion architecture

Landing-> Bronze (Raw)-> Silver-> Gold

Azure Storage account

Trendytechdltdemosa

- landing

- orders (orders1.csv)

- customers (customers1.csv)

In landing we will have raw data in csv format

Bronze- we will have data in delta format (tables created)

bronze will act like the source of truth

Data Engineers

Silver (cleaning, enrich, Merge, SCD2)

- Silver cleaned (implementing data quality checks)
- Silver merged (SCD2 , Merge)

orders (merge)

customers (SCD2)

Machine learning team / Data scientists / Data Analysts

Gold- Domain specific data (Marketing/Sales) aggregate for reporting

Business users

Trendytechdltedemosa

- landing
 - orders (orders1.csv)
 - customers (customers1.csv)

I want to create a mount point in databricks

trendytech-dlt-demo-ws1

storageAccountName = "<storage-account-name>"

storageAccountAccessKey = "<storage-account-key>"

blobContainerName = "landing"

mountPoint = "/mnt/landing/"

if not any(mount.mountPoint == mountPoint for mount in dbutils.fs.mounts()):

try:

```
dbutils.fs.mount(  
source =  
"wasbs://{}/@{}.blob.core.windows.net".format(blobContainerName,  
storageAccountName),  
mount_point = mountPoint,  
extra_configs = {'fs.azure.account.key.' + storageAccountName +  
'blob.core.windows.net': storageAccountAccessKey}  
)  
print("mount succeeded!")  
except Exception as e:  
print("mount exception", e)  
  
dbutils.fs.ls('/mnt/landing')
```

In the next session I will show how to write declarative code...

write the declarative code

trendytechdltdemosa

- landing
 - orders (orders1.csv)
 - customers (customers1.csv)

Step 1: Load the data from landing zone and move it to bronze layer

CREATE STREAMING TABLE orders_bronze

as

SELECT *,

```
input_file_name() as filename,  
current_timestamp() as load_time  
FROM cloud_files('/mnt/landing/orders', 'csv',  
map("cloudFiles.inferColumnTypes", "True"));
```

```
CREATE STREAMING TABLE customers_bronze  
as  
SELECT *,  
input_file_name() as filename,  
current_timestamp() as load_time  
FROM cloud_files('/mnt/landing/customers', 'csv',  
map("cloudFiles.inferColumnTypes", "True"));
```

Step 2: Take the data from bronze, clean it and load it to silver cleaned

```
CREATE STREAMING TABLE orders_silver_cleaned (  
CONSTRAINT valid_order EXPECT (order_id is NOT NULL) ON  
VIOLATION DROP ROW,  
CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL)  
ON VIOLATION DROP ROW  
) as  
SELECT orderid as order_id,  
orderdate as order_date,  
customerid as customer_id,  
totalamount as total_amount,  
status,  
filename as file_name,
```

load_time

FROMSTREAM(live.orders_bronze)

CREATE STREAMING TABLE customers_silver_cleaned(
CONSTRAINT valid_customer EXPECT (customer_id is NOT NULL)
ONVIOLATION DROP ROW

) as

SELECT customerid as customer_id,
customername as customer_name,
address as city,
dateofbirth as dob,
registrationdate as customer_since,
filename as file_name,
load_time
from STREAM(live.customers_bronze);

Step 3: Customers- Take the data from Silver Cleaned, handle duplicates (SCD2) and load in Silver layer

CREATE STREAMING TABLE customers_silver;
APPLY CHANGESinto live.customers_silver
from STREAM(live.customers_silver_cleaned)
keys(customer_id)
sequence by load_time
stored as scd type 2;

step 4: Orders- Take the data from silver cleaned, handle duplicates (Merge) and load in silver layer

```
CREATE STREAMING TABLE orders_silver;
```

```
APPLY CHANGES into live.orders_silver  
from STREAM(live.orders_silver_cleaned)  
keys(order_id)  
sequence by load_time;
```

step 5: Gold Layer- Take the data from Silver and perform aggregations and load to gold layer.

```
CREATE MATERIALIZED VIEW city_wise_sales_gold  
as  
SELECT city, sum(total_amount) as total_sales  
from live.orders_silver o  
join live.customers_silver c  
on o.customer_id = c.customer_id  
group by city;
```

DLT has 2 core abstractions

- Streaming table (it is a delta table) it has a stream writing to it.
it is used for ingesting huge amount of data where we cant afford to do recomputation.
- Materialized view- The results of query stored in a delta table. It is the answer to the query.

Expectations

- do not drop just a raise a warning
- drop bad records
- abort processing for a single bad record

Streaming live table- streaming table

live table- materialized view

catalog

schema (database)

table

Hive Metastore

/mnt/landing/storage

for materialize view (Enzyme)

Enzyme- set of optimization to make sure

Materialized view

